

# OPMIG-02: OpenPipeline Migration Guide: Part 2

---

**Series:** OPMIG — OpenPipeline Migration | **Notebook:** 2 of 10 | **Created:** December 2025 | **Last Updated:** 09/24/2026

## Architecture & Key Concepts

---

### Table of Contents

---

1. [Data Flow Architecture](#)
  2. [Processing Stages in Detail](#)
  3. [Pipeline Types](#)
  4. [Processor Types](#)
  5. [Dynatrace Pattern Language \(DPL\)](#)
  6. [Complete OpenPipeline Limits Reference](#)
  7. [Entity Field Availability Timeline](#)
  8. [Key Fields & Metadata](#)
  9. [Exploring Your Pipeline Configuration](#)
  10. [Understanding Processing Order](#)
  11. [Summary: Key Architecture Concepts](#)
- 

### Learning Objectives

---

By the end of this notebook, you will:

-  Understand the complete OpenPipeline data flow architecture

-  Learn detailed processing stage execution order
-  Master DPL (Dynatrace Pattern Language) fundamentals
-  **Know ALL OpenPipeline limits in comprehensive detail**
-  Understand processor types and capabilities
-  **Learn entity field availability timeline**
-  Explore your pipeline configuration with DQL

---

## Prerequisites

---

| Requirement           | Details                                                     |
|-----------------------|-------------------------------------------------------------|
| Dynatrace Environment | SaaS or Managed with Grail enabled                          |
| API Access            | <code>logs.read</code> token scope for DQL validation       |
| Knowledge             | OPMIG-01 or familiarity with Classic log ingestion concepts |

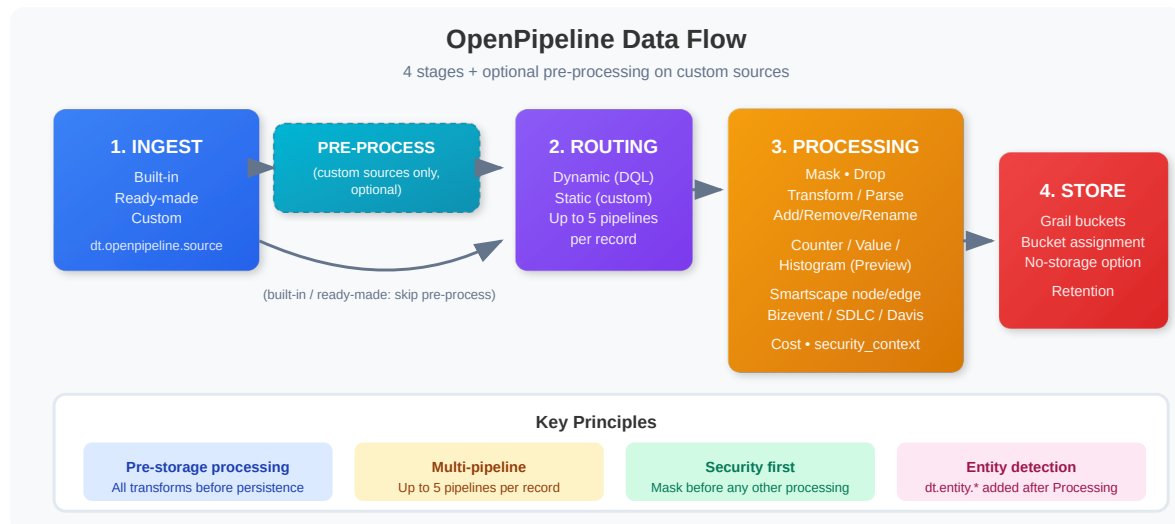
---

## Data Flow Architecture

---

Understanding how data flows through OpenPipeline is essential for designing effective pipelines.

# Complete Data Flow



**Stage model (verified 09/24/2026):** data flows Ingest → Routing → pipeline → Storage, with optional pre-processing on custom sources. Inside a pipeline, per *Processing in OpenPipeline*, "The sequence of stages is fixed for all pipelines and cannot be modified." Masking, dropping, parsing and transformation are processors in the first stage, **Processing**; **Bucket assignment** comes seventh, and **Metric extraction**, **Davis** and **Data extraction** run after it. The diagram's Processing box stands for that whole stage sequence.

**Sources:** [Processing in OpenPipeline \(DT docs\)](#) — stage table, in execution order.

## Key Principles

1. **Pre-storage Processing:** All transformations happen BEFORE data is written to Grail.
2. **Order Matters:** Processors execute in the order defined within each pipeline.
3. **Multi-pipeline Support:** A single record can be processed by up to 5 pipelines.
4. **Entity Detection:** Entity fields ( `dt.entity.*` ) are added AFTER the Processing stage.
5. **Immutable Storage:** Once stored, data cannot be modified (mask early!).

# Processing Stages in Detail

## Stage 1: Ingest

Records enter via an ingest source. The source is recorded as `dt.openpipeline.source`.

| Source Type | Owner                    | Pre-processing | Routing           |
|-------------|--------------------------|----------------|-------------------|
| Built-in    | OpenPipeline (view-only) | No             | Dynamic only      |
| Ready-made  | Extensions               | No             | Dynamic only      |
| Custom      | User-defined             | Yes (optional) | Dynamic or static |

## Stage 2: Pre-processing (*optional, custom sources only*)

A DQL transform applied to records from a custom source *before* routing. Use this to normalize raw data from third-party shippers into a common shape that all downstream pipelines can consume.

## Stage 3: Routing

The routing stage determines which pipeline(s) process each incoming record.

| Aspect        | Description                                                                             |
|---------------|-----------------------------------------------------------------------------------------|
| Purpose       | Match incoming data to appropriate pipelines                                            |
| Timing        | Before any pipeline processing                                                          |
| Configuration | Dynamic routing rules with matching conditions, or static assignment for custom sources |
| Fallback      | Unmatched data goes to the default pipeline                                             |

### Matching Condition Examples:

```
k8s.namespace.name == "production"
log.source == "nginx"
contains(content, "payment")
dt.openpipeline.source == "oneagent"
```

## Stage 4: Processing

All in-pipeline work happens here. Processors execute in the order defined in the pipeline. Functional groups within Processing:

| Group                       | Processors                                                                                             | Purpose                                         |
|-----------------------------|--------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| Masking                     | DQL with <code>replacePattern</code>                                                                   | Redact PII before anything else                 |
| Filtering                   | Drop record                                                                                            | Remove unwanted records early                   |
| Field & record manipulation | Add/Remove/Rename fields, DQL, Parse, Technology                                                       | Transform and enrich                            |
| Metric extraction           | Counter, Value, Histogram (Preview); sampling-aware variants for spans                                 | Generate timeseries from records                |
| Smartscape topology         | Smartscape node, Smartscape edge                                                                       | Populate the Smartscape topology graph          |
| Event extraction            | Business event, SDLC event, Davis event                                                                | Convert records into events                     |
| Cost & security             | DPS Cost Allocation — Cost Center, DPS Cost Allocation — Product, Set <code>dt.security_context</code> | Attribute cost / apply record-level access      |
| Storage assignment          | Bucket assignment, No storage assignment                                                               | Place the record in a bucket, or skip retention |

Recommended order *within* Processing: mask → drop → transform/parse → extract → assign cost/security → bucket assignment.

## Stage 5: Storage

The storage stage routes data to Grail buckets. Use the **No storage assignment** processor to skip retention entirely (useful when a record is only needed to extract a metric or event).

| Aspect         | Description                                                  |
|----------------|--------------------------------------------------------------|
| Bucket Routing | Direct data to specific buckets                              |
| Retention      | Different retention periods per bucket                       |
| Cost Control   | Route high-volume data to shorter retention or to no-storage |

---

## Pipeline Types

---

OpenPipeline supports three types of pipelines:

### 1. Default Pipeline

- **Purpose:** Handles data that doesn't match any custom pipeline
- **Configuration:** Minimal processing, default bucket
- **Modification:** Can add processors but cannot delete

### 2. Custom Pipelines

- **Purpose:** Targeted processing for specific data sources or patterns
- **Configuration:** Full control over all processing stages
- **Routing:** Requires matching conditions in dynamic routing

### 3. Built-in Pipelines

- **Purpose:** Pre-configured for common technologies
- **Examples:** Kubernetes logs, AWS logs, Azure logs
- **Modification:** Can extend but not fundamentally change



**Best Practice:** Create focused pipelines for specific use cases rather than one large pipeline with complex conditionals.

---

# Processor Types

## DQL Processor

The DQL processor uses DQL commands to transform data. Available commands:

| Command                   | Purpose                   | Example                                           |
|---------------------------|---------------------------|---------------------------------------------------|
| <code>fieldsAdd</code>    | Add new fields            | <code>fieldsAdd environment = "prod"</code>       |
| <code>fieldsRemove</code> | Remove fields             | <code>fieldsRemove sensitive_field</code>         |
| <code>fieldsRename</code> | Rename fields             | <code>fieldsRename old_name = new_name</code>     |
| <code>parse</code>        | Extract with DPL patterns | <code>parse content, "LD:prefix INT:count"</code> |

### fieldsAdd Examples:

```
// Static value
| fieldsAdd environment = "production"

// Conditional value
| fieldsAdd severity = if(loglevel == "ERROR", "critical", else: "normal")

// Computed value
| fieldsAdd message_length = stringLength(content)

// From existing field
| fieldsAdd short_host = substring(host.name, 0, 10)
```

## Drop Processor

Removes records matching specified conditions:

```
Matching Condition: loglevel == "DEBUG"
Result: All DEBUG logs are dropped before storage
```

## Technology Processors

Built-in parsers for common log formats:

| Technology | Parsed Fields                                |
|------------|----------------------------------------------|
| Apache     | client_ip, method, path, status, bytes       |
| Nginx      | Similar to Apache with nginx-specific fields |
| JSON       | Flattens JSON structure to fields            |
| Syslog     | facility, severity, hostname, message        |

## Dynatrace Pattern Language (DPL)

DPL is a powerful pattern matching language used in the `parse` command.

### Core Matchers

| Matcher   | Description              | Example Match            |
|-----------|--------------------------|--------------------------|
| INT       | Integer number           | 42 , -17                 |
| LONG      | Long integer             | 1234567890123            |
| DOUBLE    | Decimal number           | 3.14 , -0.5              |
| IPADDR    | IPv4 or IPv6 address     | 192.168.1.1 , ::1        |
| IPV4ADDR  | IPv4 address only        | 10.0.0.1                 |
| IPV6ADDR  | IPv6 address only        | 2001:db8::1              |
| TIMESTAMP | Timestamp with format    | 2024-01-15T10:30:00Z     |
| LD        | Line data (to delimiter) | Any text until delimiter |
| DATA      | Any data (greedy)        | Consumes remaining text  |
| SPACE     | Whitespace               | Spaces, tabs             |
| NSPACE    | Non-whitespace           | Any non-space chars      |
| WORD      | Word characters          | hello , user123          |
| JSON      | JSON structure           | {"key": "value"}         |

| Matcher | Description | Example Match   |
|---------|-------------|-----------------|
| EOL     | End of line | Line terminator |

## Pattern Syntax

| Element         | Syntax                         | Description                |
|-----------------|--------------------------------|----------------------------|
| Export to field | <code>MATCHER:fieldname</code> | Extract and name the field |
| Match only      | <code>MATCHER</code>           | Match but don't extract    |
| Optional        | <code>MATCHER?</code>          | Matcher is optional        |
| Literal         | <code>'exact text'</code>      | Match literal string       |
| Alternatives    | <code>('opt1'\ 'opt2')</code>  | Match either option        |
| Quantifier      | <code>MATCHER{2,5}</code>      | Match 2-5 times            |

**Quantifier caveat:** `{n,m}` is not universal. `INT` accepts only `*` and `+` — `INT{3}` is rejected. For fixed-width digit runs use a character class: `[0-9]{3}`.

## DPL Examples

```
// Extract IP and port from log
| parse content, "IPADDR:client_ip ':' INT:port"

// Extract user ID with flexible prefix
| parse content, "('user='| 'userId='| 'user_id=')LD:user_id"

// Parse Apache-style log
| parse content, "IPADDR:client_ip SPACE '-' SPACE LD:user SPACE '['
LD:timestamp ']"

// Extract JSON payload
| parse content, "LD JSON:payload"

// Parse with optional port
| parse content, "IPADDR:ip (':' INT:port)?"

// Extract error code
| parse content, "'error_code=' INT:error_code"
```

## Timestamp Format Patterns

| Symbol | Meaning          | Example |
|--------|------------------|---------|
| yyyy   | Year (4 digits)  | 2024    |
| MM     | Month (01-12)    | 01      |
| dd     | Day (01-31)      | 15      |
| HH     | Hour 24h (00-23) | 14      |
| mm     | Minute (00-59)   | 30      |
| ss     | Second (00-59)   | 45      |
| SSS    | Milliseconds     | 123     |

# Complete OpenPipeline Limits Reference

This comprehensive reference contains ALL OpenPipeline limits you need to know for planning your migration.

## Data Size & Volume Limits

| Limit                               | Value          | Behavior When Exceeded                     |
|-------------------------------------|----------------|--------------------------------------------|
| Max record size (after processing)  | 16 MB          | Record is <b>dropped</b> , logged in audit |
| Max record size (before processing) | 10 MB          | Ingestion rejected with 413 error          |
| Working memory per record           | 16 MB          | Processing fails, record dropped           |
| Log attribute size                  | 32 KB          | Attribute value <b>truncated</b>           |
| Max field name length               | 255 characters | Field creation fails                       |
| Max string field length             | 32 KB          | Content truncated                          |
| Max array size                      | 1000 elements  | Array truncated                            |
| Max nesting depth (JSON)            | 10 levels      | Deeper levels flattened                    |

## Processing Limits

| Limit                          | Value       | Impact                                                                                                                      |
|--------------------------------|-------------|-----------------------------------------------------------------------------------------------------------------------------|
| Max pipelines per record       | 5           | A record can be processed by up to 5 different pipelines. After 5, data extraction stops but the record is still persisted. |
| Max processors per pipeline    | 1,000       | Cannot add more processors to pipeline                                                                                      |
| Max DQL commands per processor | 10 commands | Split complex logic into multiple processors                                                                                |

| Limit                              | Value          | Impact                             |
|------------------------------------|----------------|------------------------------------|
| Max parse operations per processor | 100 patterns   | Create additional parse processors |
| Max fields per record              | 500 fields     | Additional fields ignored          |
| Processing timeout per record      | 30 seconds     | Record dropped if exceeded         |
| Max processor name length          | 100 characters | Validation error                   |

## Timestamp Constraints

| Data Type       | Accepted Range                     | Records Outside Range |
|-----------------|------------------------------------|-----------------------|
| Logs            | 24 hours past to 10 minutes future | Dropped               |
| Spans           | 60 minutes past (end time)         | Dropped               |
| Events          | 24 hours past to 10 minutes future | Dropped               |
| Business Events | 24 hours past to 10 minutes future | Dropped               |
| Metrics         | 1 hour past to 1 minute future     | Dropped               |

 **Critical:** Historical data imports require workarounds. Contact Dynatrace support for backfilling options.

## Pipeline Group Limits (*per group*)

| Limit                      | Value | Notes                                 |
|----------------------------|-------|---------------------------------------|
| Pipeline slots per group   | 10    | Per <a href="#">/reference/limits</a> |
| Member pipelines per group | 1,000 | Per <a href="#">/reference/limits</a> |

## Pipeline & Routing Limits

| Limit                    | Value          | Scope                                       |
|--------------------------|----------------|---------------------------------------------|
| Max custom pipelines     | 100 pipelines  | Per configuration scope (logs, spans, etc.) |
| Max dynamic routes       | 3,000 routes   | Per configuration scope                     |
| Max conditions per route | 10 conditions  | Combine with AND/OR operators               |
| Max pipeline name length | 100 characters | Validation error                            |
| Max route name length    | 100 characters | Validation error                            |

## Extraction Limits

| Limit                                 | Value          | Notes                            |
|---------------------------------------|----------------|----------------------------------|
| Max metric extractions per pipeline   | 10             | Value + counter metrics combined |
| Max event extractions per pipeline    | 5              | All event types combined         |
| Max bizevent extractions per pipeline | 3              | Business events only             |
| Max dimensions per metric             | 10 dimensions  | Keep cardinality low             |
| Metric key length                     | 250 characters | Validation error                 |
| Event type name length                | 100 characters | Validation error                 |

## DQL & DPL Limits

| Limit                               | Value                                                                                       | Context                                                  |
|-------------------------------------|---------------------------------------------------------------------------------------------|----------------------------------------------------------|
| DQL processor script length         | 8,192 characters                                                                            | Per <a href="#">/reference/limits</a> (May 2026 doc)     |
| Processor matching condition length | 4,096 characters (Settings API);<br>1,500 for legacy Configurations API / Classic pipelines | Per <a href="#">/reference/limits</a> (Jun 17, 2026 doc) |

| Limit                         | Value           | Context                               |
|-------------------------------|-----------------|---------------------------------------|
| Max DPL pattern length        | 4 KB            | Per parse pattern                     |
| Max captured groups per parse | 100 groups      | Use multiple parse operations         |
| Max alternatives in pattern   | 50 alternatives | <code>(opt1\ opt2\ ...\ opt50)</code> |

## Bucket & Storage Limits

| Limit                | Value                                                                      | Notes                                                                                                                                                                                               |
|----------------------|----------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Max custom buckets   | 80 by default; 250 from SaaS 1.346 (staged tenant rollout from 08/25/2026) | Per environment. The upgrade guide still states 80, the Grail <i>organize data</i> page and the 1.346 release note state 250 — check which default your tenant has before planning to either number |
| Min retention period | 1 day                                                                      | Per bucket                                                                                                                                                                                          |
| Max retention period | 2555 days (~7 years)                                                       | Per bucket                                                                                                                                                                                          |
| Bucket name length   | 100 characters                                                             | Alphanumeric + underscore                                                                                                                                                                           |

**Sources:** [Data partitioning — upgrade best practices \(DT docs\)](#) — “The default bucket limit per environment is 80 buckets, which is typically sufficient for up to 5 TB/day per table.”; [Organize data \(DT docs\)](#) — “The default limit per environment is 250 custom buckets.”; [SaaS 1.346 \(DT docs\)](#) — “Dynatrace now supports up to 250 custom Grail buckets by default”.

## Rate Limits

| Endpoint       | Limit             | Per       |
|----------------|-------------------|-----------|
| Log Ingest API | 500 requests/min  | Per token |
| OTLP Endpoint  | 1000 requests/min | Per token |
| Config API     | 100 requests/min  | Per token |

## Field & Matching Restrictions

**Read-Only Fields** (Cannot be modified):

```
dt.ingest.*      - Ingestion metadata
dt.openpipeline.* - Pipeline processing metadata
dt.retain.*      - Retention information
dt.system.*      - System metadata (bucket, etc.)
timestamp        - Record timestamp (can parse new, not modify original)
```

**Reserved Field Prefixes** (Cannot create):

```
dt.*      - Reserved for Dynatrace
dynatrace.* - Reserved for Dynatrace
```

**Entity Fields** (Available ONLY after Processing stage):

```
dt.entity.service
dt.entity.host
dt.entity.process_group
dt.entity.process_group_instance
dt.entity.kubernetes_cluster
dt.entity.cloud_application
dt.entity.cloud_application_namespace
```



**Design Pattern:** Entity fields are added by Dynatrace AFTER the Processing stage. You cannot:

- Use them in routing conditions
- Use them in matching conditions during processing
- Modify them with fieldsAdd/fieldsRename

You CAN use them in:

- Extraction stage (metrics, events)
- Storage stage (bucket routing based on entity)

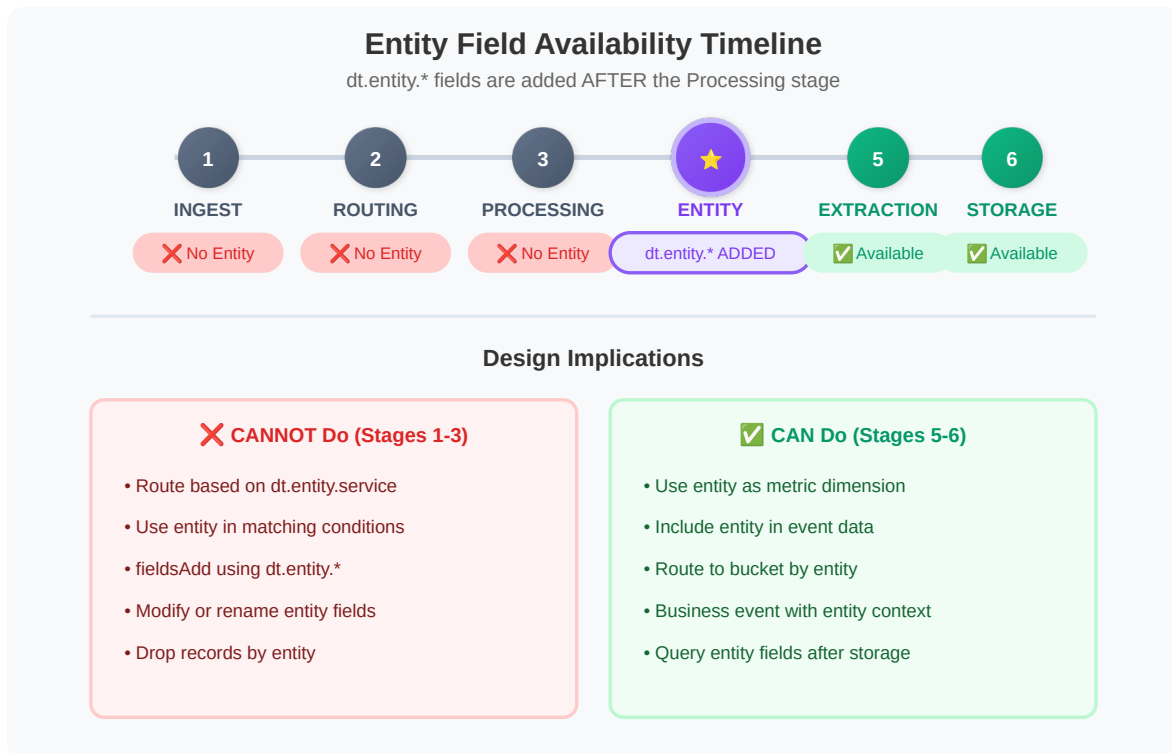
## Workarounds for Common Limit Issues

| Problem                | Workaround                                                |
|------------------------|-----------------------------------------------------------|
| >50 processors needed  | Split into multiple pipelines, use multi-pipeline routing |
| >10 DQL commands       | Break into multiple processors (order matters!)           |
| >100 parse patterns    | Use multiple parse processors in sequence                 |
| >16MB after processing | Drop unnecessary fields, reduce field sizes               |
| >5 pipelines needed    | Consolidate processing logic, use conditional processors  |
| >10 metric dimensions  | Reduce cardinality, use separate metrics                  |

## Entity Field Availability Timeline

Understanding WHEN entity fields become available is critical for pipeline design.

## Processing Stage Timeline



## Practical Examples

### ✗ WRONG - Cannot route based on entity

Dynamic Route:

```
Condition: dt.entity.service == "SERVICE-123" ✗ FAILS
```

Why: Entity fields not available during routing

### ✗ WRONG - Cannot use entity in processing

DQL Processor:

```
fieldsAdd service_name = dt.entity.service ✗ FAILS
```

Why: Entity fields not available during processing

## ✓ CORRECT - Use entity in metric extraction

```
Metric Extraction:  
  Key: log.request.duration  
  Value: duration_ms  
  Dimensions: dt.entity.service, dt.entity.host ✓ WORKS  
  
Why: Entity fields available in extraction stage
```

## ✓ CORRECT - Use entity for bucket routing

```
Bucket Routing (in Storage stage):  
  Condition: dt.entity.service == "CRITICAL-SERVICE"  
  Bucket: critical_logs ✓ WORKS  
  
Why: Entity fields available in storage stage
```

## Design Patterns

### Pattern 1: Route by source, extract metrics by entity

```
Routing: log.source == "payment-service" (uses source field)  
Processing: Parse payment data  
Extraction: Create metric with dt.entity.service dimension ✓
```

### Pattern 2: Enrich with custom field, then use entity

```
Processing: fieldsAdd app_tier = "frontend" (custom field)  
Extraction: Dimensions: app_tier, dt.entity.service ✓
```

### Pattern 3: Cannot mix entity with routing

```
✗ Route based on entity → Use workaround:  
  - Route based on source/content patterns instead  
  - Use service name string (not entity ID) if available
```

---

---

# Key Fields & Metadata

## OpenPipeline-Specific Fields

| Field                                  | Description                       | Example Value                                                       |
|----------------------------------------|-----------------------------------|---------------------------------------------------------------------|
| <code>dt.openpipeline.source</code>    | Data source identifier            | <code>oneagent</code> , <code>generic</code> ,<br><code>otlp</code> |
| <code>dt.openpipeline.pipelines</code> | Pipeline(s) that processed record | <code>["custom-pipeline-1"]</code>                                  |
| <code>dt.system.bucket</code>          | Grail storage bucket              | <code>default_logs</code> ,<br><code>custom_logs</code>             |

## Log Fields

| Field                     | Description                             |
|---------------------------|-----------------------------------------|
| <code>timestamp</code>    | Log timestamp (primary time field)      |
| <code>content</code>      | Log message content                     |
| <code>loglevel</code>     | Log level (ERROR, WARN, INFO, DEBUG)    |
| <code>status</code>       | Status string (alternative to loglevel) |
| <code>log.source</code>   | Log source identifier                   |
| <code>log.iostream</code> | Stream type (stdout, stderr)            |

## Entity Context Fields

| Field                                         | Description                |
|-----------------------------------------------|----------------------------|
| <code>dt.entity.host</code>                   | Host entity ID             |
| <code>dt.entity.process_group</code>          | Process group entity ID    |
| <code>dt.entity.process_group_instance</code> | Process instance entity ID |
| <code>dt.entity.service</code>                | Service entity ID          |

| Field                                | Description             |
|--------------------------------------|-------------------------|
| <code>host.name</code>               | Host name (string)      |
| <code>process.executable.name</code> | Process executable name |

 **Important:** Entity fields ( `dt.entity.*` ) are added AFTER the Processing stage. You cannot use them in routing or processing conditions.

## Exploring Your Pipeline Configuration

Use these queries to understand how your environment is configured.

```
// View data sources currently sending to OpenPipeline
// Shows the distribution of data by ingestion source
fetch logs, from: now() - 24h
| summarize {record_count = count()}, by: {dt.openpipeline.source}
| sort record_count desc
```

```
// Analyze which pipelines are processing your logs
// Helps verify routing is working correctly
fetch logs, from: now() - 24h
| filter isNotNull(dt.openpipeline.pipelines)
| summarize {record_count = count()}, by: {dt.openpipeline.pipelines}
| sort record_count desc
```

```
// Check bucket distribution for stored logs
// Verify data is routing to expected buckets
fetch logs, from: now() - 24h
| summarize {record_count = count()}, by: {dt.system.bucket}
| sort record_count desc
```

```
// Analyze pipeline processing by source and pipeline
// Shows the relationship between sources and pipelines
fetch logs, from: now() - 24h
| filter isNotNull(dt.openpipeline.pipelines)
| summarize {record_count = count()}, by: {dt.openpipeline.source,
dt.openpipeline.pipelines}
| sort record_count desc
| limit 25
```

```
// Check for span processing through OpenPipeline
// Spans also flow through OpenPipeline
fetch spans, from: now() - 24h
| summarize {span_count = count()}, by: {dt.openpipeline.source}
| sort span_count desc
```

```
// View pipeline processing over time
// Helps identify volume patterns and processing trends
fetch logs, from: now() - 24h
| filter isNotNull(dt.openpipeline.pipelines)
| makeTimeseries {record_count = count()}, by: {dt.openpipeline.pipelines},
interval: 1h
```

```
// Identify logs going to default pipeline (may need custom routing)
// High volume in default may indicate missing routing rules
fetch logs, from: now() - 24h
| filter isNull(dt.openpipeline.pipelines) OR dt.openpipeline.pipelines == "
[]"
| summarize {unrouted_count = count()}, by: {log.source}
| sort unrouted_count desc
| limit 20
```

---

## Understanding Processing Order

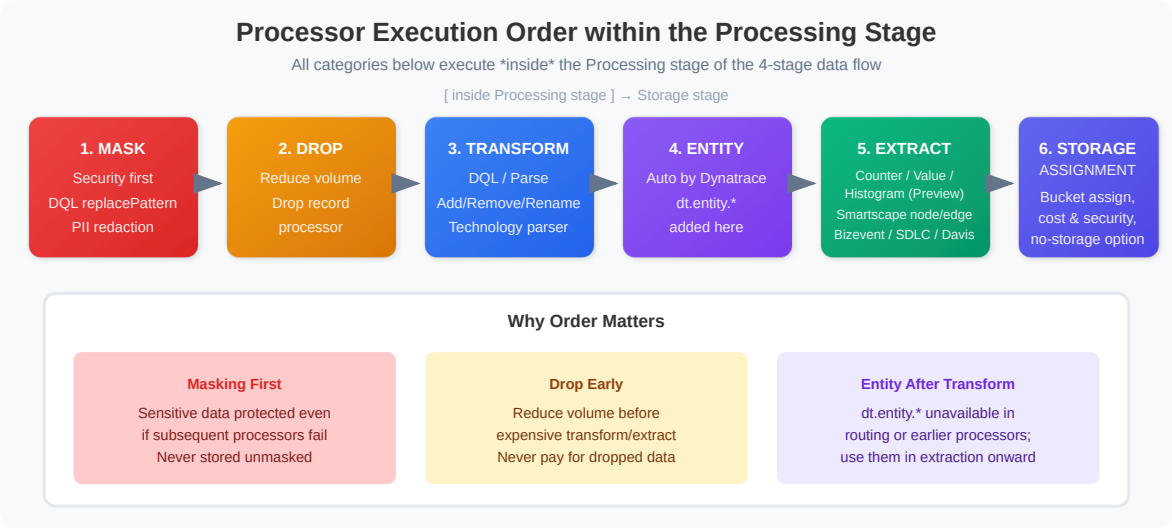
---

Two orders are in play, and only one of them is yours to choose:

1. **The stage sequence is fixed.** Per *Processing in OpenPipeline*, "The sequence of stages is fixed for all pipelines and cannot be modified." Every pipeline runs **Processing** → Smartscape node → Smartscape edge → Permission → Product allocation → Cost allocation → **Bucket assignment** → **Metric extraction** → **Davis** →

**Data extraction.** Masking, dropping, parsing and transformation are all processors in the first stage; there is no separate masking or filtering stage, and the extraction stages run *after* bucket assignment.

2. **Processor order within a stage is the order you configure** — “each processor output becomes the input for the next one.”



💡 **Tip:** Masking is not automatically first. Inside the Processing stage, place masking processors **before** any processor that copies, parses or extracts from the sensitive field — a processor listed ahead of the masking processor sees the raw value. Records reach Bucket assignment and storage only after the whole Processing stage has run. The diagram groups processors by purpose; the execution order the platform enforces is the stage sequence above.

**Sources:** [Processing in OpenPipeline \(DT docs\)](#) — stage table, in execution order; “The processor order in the stage; each processor output becomes the input for the next one.”

## Summary: Key Architecture Concepts

| Concept     | Key Points                                                               |
|-------------|--------------------------------------------------------------------------|
| Data Flow   | Ingest → (Pre-process) → Route → pipeline (fixed stage sequence) → Store |
| Pre-storage | All processing happens before data is persisted                          |

| Concept                 | Key Points                                                                                                                                                                                                               |
|-------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Source Types</b>     | Built-in / Ready-made / Custom — only custom sources support pre-processing and static routing                                                                                                                           |
| <b>Routing</b>          | Dynamic (DQL matcher) or static (custom sources only)                                                                                                                                                                    |
| <b>Processing Order</b> | Fixed stages: Processing (mask, drop, parse, transform — in the order you list them) → Smartscape node/edge → Permission → Product and cost allocation → Bucket assignment → Metric extraction → Davis → Data extraction |
| <b>Entity Detection</b> | Happens AFTER processing, BEFORE most extraction processors run                                                                                                                                                          |
| <b>Multi-pipeline</b>   | One record can be processed by up to 5 pipelines ( <a href="#">/reference/limits</a> )                                                                                                                                   |
| <b>DPL</b>              | Powerful pattern language for parsing                                                                                                                                                                                    |
| <b>Buckets</b>          | Control retention and cost at the storage assignment step; or skip with No storage assignment                                                                                                                            |

## Next Steps

Now that you understand OpenPipeline architecture, continue with:

| Notebook        | Focus Area                           |
|-----------------|--------------------------------------|
| <b>OPMIG-03</b> | Migration Assessment & Planning      |
| <b>OPMIG-04</b> | Pipeline Configuration Fundamentals  |
| <b>OPMIG-05</b> | Routing & Bucket Management          |
| <b>OPMIG-06</b> | Processing, Parsing & Transformation |

# References

---

- [OpenPipeline Data Flow](#)
  - [OpenPipeline Processing](#)
  - [Dynatrace Pattern Language](#)
  - [OpenPipeline Limits](#)
  - [DPL Architect Tool](#)
- 

*Last Updated: September 24, 2026*

---

*This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.*